

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине «Модели решения задач в И С»

Тема: «Бинарная классификация логических функций и переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26

Рубцов Д.А.

Проверил:

Андренко К.В.

Брест 2026

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для своей логической функции (2- примеров).
 2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте `np.random.seed(42)` для воспроизводимости).
 3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
 - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
 - Реализовать правильный градиент для BCE + сигмоида:
 4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
 - А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)
 - Б. MSE + адаптивный шаг (из ЛР №4)
 - В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
 - Г. BCE + адаптивный шаг (по формуле из ЛР №2)
- Критерий остановки для всех – суммарная ошибка $E_s \leq E_e$ (возьмите E_e из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
 - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
 - Зафиксировать количество эпох до достижения критерия остановки.
 - После обучения посчитать ассигасу на обучающей и тестовой выборках.
 - Проверить, сколько процентов из всей таблицы истинности (2- примеров) сеть предсказывает правильно.
 6. Построить один общий график зависимости суммарной ошибки E_s от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).
 7. Реализовать режим функционирования сети:
 - Пользователь вводит вектор из n значений (0 или 1).
 - Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
 - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
 8. Написать вывод (обязательно ответить на следующие пункты):
 - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
 - Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
 - Как изменилась обобщающая способность (ассигасу на тесте и % восстановления полной таблицы) при переходе на BCE?
 - Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?
 - Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Вариант 13:

13	8	OR
----	---	----

Код программы:

```
import numpy as np
import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt

X_full = np.array([list(map(int, format(i, '08b')) for i in range(256)], dtype=float)
e_full = (np.sum(X_full, axis=1) > 0).astype(float).reshape(-1, 1)
```

```

np.random.seed(42)
idx = np.random.permutation(len(X_full))
X_train, e_train = X_full[idx[:204]], e_full[idx[:204]]
X_test, e_test = X_full[idx[204:]], e_full[idx[204:]]

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def predict(w, x, T):
    return sigmoid(x @ w - T)

def bce_loss(y, t):
    y = np.clip(y, 1e-15, 1 - 1e-15)
    return -np.mean(t * np.log(y) + (1 - t) * np.log(1 - y))

def get_accuracy(w, T, X, e):
    preds = (predict(w, X, T) > 0.5).astype(int)
    return np.mean(preds == e) * 100

def train(X, e, X_tst, e_tst, mode='MSE', alpha=0.02, adaptive=False, tol=1e-6):
    N = X.shape[0]
    w = np.zeros((X.shape[1], 1))
    T = 0.0
    train_errors = []
    test_errors = []
    prev_err = float('inf')

    for epoch in range(5000):
        perm = np.random.permutation(N)
        Xs, es = X[perm], e[perm]
        for i in range(N):
            xi, ti = Xs[i:i+1], es[i]
            y = predict(w, xi, T)

            err = (y - ti) * y * (1 - y) if mode == 'MSE' else (y - ti)

            a = 1 / (1 + np.sum(xi**2)) if adaptive else alpha
            w -= a * (xi.T @ err)
            T -= a * (-err[0, 0])

        y_train = predict(w, X, T)
        y_test = predict(w, X_tst, T)

        if mode == 'MSE':
            curr_train_err = np.mean((y_train - e)**2)
            curr_test_err = np.mean((y_test - e_tst)**2)
        else:
            curr_train_err = bce_loss(y_train, e)
            curr_test_err = bce_loss(y_test, e_tst)

        train_errors.append(curr_train_err)
        test_errors.append(curr_test_err)

        if epoch > 30 and abs(prev_err - curr_train_err) < tol:
            break
        prev_err = curr_train_err

    return w, T, train_errors, test_errors, epoch + 1

models_cfg = [
    ('MSE', 0.02, False), ('MSE', 0.02, True),
    ('BCE', 0.02, False), ('BCE', 0.02, True)
]

results = {}
plt.figure(figsize=(10, 6))

for mode, alpha, adapt in models_cfg:
    name = f"{mode} {'adapt' if adapt else 'fixed'}"
    w, T, t_errs, v_errs, epochs = train(X_train, e_train, X_test, e_test, mode, alpha, adapt)

    acc_train = get_accuracy(w, T, X_train, e_train)
    acc_test = get_accuracy(w, T, X_test, e_test)
    acc_full = get_accuracy(w, T, X_full, e_full)

    results[name] = (w, T, acc_full)

    print(f"--- {name} ---")
    print(f"Завершено на эпохе: {epochs}")

```

```

print(f"Финальная ошибка Train: {t_errs[-1]:.8f}")
print(f"Финальная ошибка Test: {v_errs[-1]:.8f}")
print(f"Accuracy Train: {acc_train:.2f}%")
print(f"Accuracy Test: {acc_test:.2f}%")
print(f"Процент всей таблицы: {acc_full:.2f}%\n")

plt.plot(t_errs, label=f"Train {name}")

plt.yscale('log')
plt.title("Кривые обучения для всех конфигураций")
plt.xlabel("Эпоха")
plt.ylabel("Значение функции потерь")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

best_model_name = max(results, key=lambda k: results[k][2])
best_w, best_T, _ = results[best_model_name]

print(f"Используется лучшая модель: {best_model_name}")
print("Введите 8 бит через пробел или exit для выхода")

while True:
    s = input("x → ").strip()
    if not s or s.lower() == 'exit': break
    try:
        vals = list(map(int, s.split()))
        if len(vals) != 8:
            print("Введите ровно 8 цифр (0 или 1)")
            continue

        x_inp = np.array([vals], dtype=float)
        y_prob = predict(best_w, x_inp, best_T)[0, 0]
        cls_pred = 1 if y_prob > 0.5 else 0

        actual_logic = 1 if sum(vals) > 0 else 0
        status = "Совпадает" if cls_pred == actual_logic else "Расхождение"

        print(f" Вероятность (y) = {y_prob:.4f}")
        print(f" Класс = {cls_pred} | {status}\n")
    except Exception as e:
        print(f"Ошибка ввода: {e}")

```

Результат программы:

```

--- MSE fixed ---
Завершено на эпохе: 296
Финальная ошибка Train: 0.00393689
Финальная ошибка Test: 0.00040742
Accuracy Train: 99.51%
Accuracy Test: 100.00%
Процент всей таблицы: 99.61%

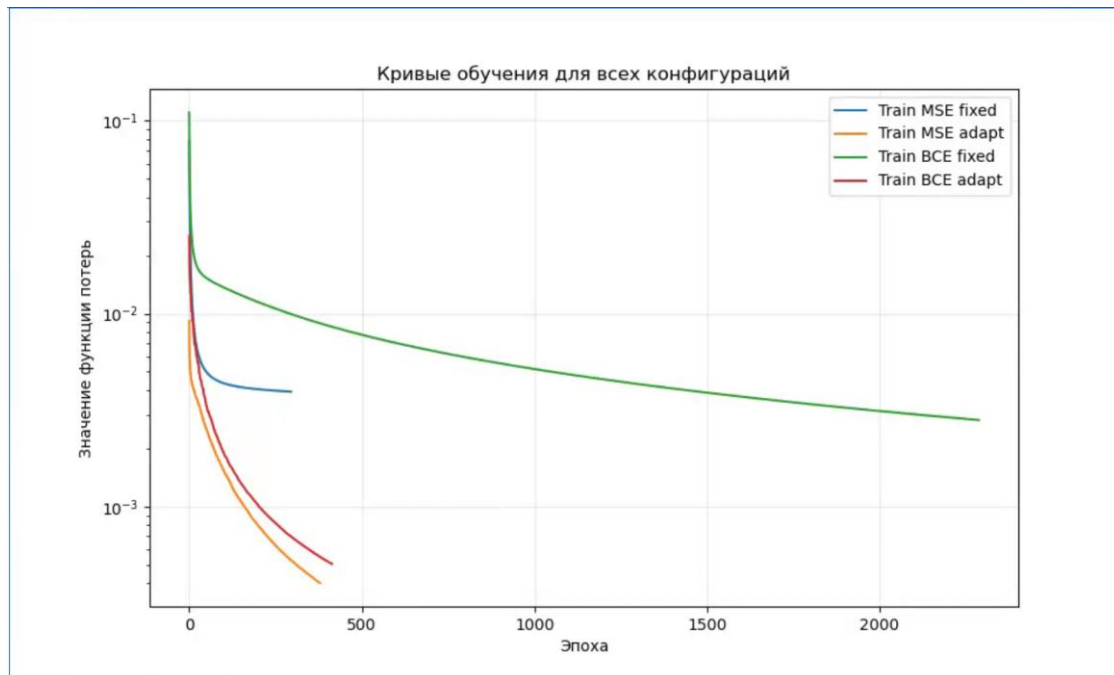
--- MSE adapt ---
Завершено на эпохе: 380
Финальная ошибка Train: 0.00040126
Финальная ошибка Test: 0.00383725
Accuracy Train: 100.00%
Accuracy Test: 100.00%
Процент всей таблицы: 100.00%

--- BCE fixed ---
Завершено на эпохе: 2288
Финальная ошибка Train: 0.00280678
Финальная ошибка Test: 0.00271594
Accuracy Train: 100.00%
Accuracy Test: 100.00%
Процент всей таблицы: 100.00%

--- BCE adapt ---
Завершено на эпохе: 414
Финальная ошибка Train: 0.00050420

```

Финальная ошибка Test: 0.01311705
Accuracy Train: 100.00%
Accuracy Test: 100.00%
Процент всей таблицы: 100.00%



На графике видно что MSE с фиксированным шагом обучения сходится за самое меньшее количество эпох (296) но в тоже время показатели ошибки десятикратно выше (≈ 0.004) по сравнению с BCE и MSE с адаптивным шагом (≈ 0.0004) при этом скорость их сходимости меньше $\approx 20\%$ что делает их наиболее лучшими вариантами в соотношении количества эпох и ошибки.

Используется лучшая модель: MSE adapt
Введите 8 бит через пробел или exit для выхода
x → 1 0 0 0 0 0 0 0
Вероятность (y) = 0.9138
Класс = 1 | Совпадает

x → 0 0 0 0 0 0 0 1
Вероятность (y) = 0.5536
Класс = 1 | Совпадает

x → 0 0 0 0 0 0 0 0
Вероятность (y) = 0.1733
Класс = 0 | Совпадает

1. Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?

В данной конфигурации быстрее всего сошелся метод MSE с фиксированным шагом (296 эпох). Метод BCE с фиксированным шагом показал самую низкую скорость сходимости (2288 эпох), что в ~ 7.7 раз медленнее, чем MSE fixed. Однако при

использовании адаптивного шага скорость BCE значительно возрастает (414 эпох), становясь сопоставимой с результатами MSE.

2. Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?

Теоретическое преимущество BCE заключается в форме её градиента. В отличие от MSE, где в формулу градиента входит множитель $y(1-y)$, в BCE этот множитель сокращается.

3. Как изменилась обобщающая способность (accuracy на тесте и % восстановления полной таблицы) при переходе на BCE?

Обобщающая способность при переходе на BCE осталась на максимальном уровне.

- MSE fixed восстановил таблицу на 99.61%.
- MSE adapt восстанавливает 100%
- Все конфигурации BCE (и fixed, и adapt) обеспечили 100% точность (accuracy) как на тестовой выборке, так и при восстановлении полной таблицы истинности. Это говорит о том, что BCE позволяет найти более надежные значения весов для четкого разделения классов.

4. Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?

Адаптивный шаг оказал критически положительное влияние на BCE. Скорость обучения увеличилась более чем в 5.5 раз (с 2288 до 414 эпох). Это связано с тем, что адаптивный шаг позволяет делать более крупные изменения весов на начальных этапах и в моменты больших ошибок, в то время как фиксированный малый шаг $\alpha=0.02$ заставляет BCE сходиться крайне медленно из-за специфики логарифмической функции потерь.

5. Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Для линейных функций таких как OR/AND достаточно однослойного персептрона, но для не линейных логических функций таких как XOR нужно использовать многослойных персептрон с одним скрытым слоем.

Вывод: Реализовал однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провел сравнение BCE с MSE по скорости сходимости, качеству обучения и обобщающей способности. Исследовал влияние адаптивного шага обучения на BCE.